

## Step 1:

The screenshot shows the AWS Management Console. The top section displays 'Instances (1/2)' with a table of two running instances. The bottom section shows 'Inbound rules (5)' with a table of five rules.

**Instances (1/2)**

| Name           | Instance ID         | Instance state | Instance type | Status check      | Availability Zone |
|----------------|---------------------|----------------|---------------|-------------------|-------------------|
| xfusion-ec2    | i-001e3a5dd2f6d5720 | Running        | t2.micro      | 2/2 checks passed | us-east-1a        |
| Myinstance_eks | i-094bed920358b5ec6 | Running        | t3.medium     | 3/3 checks passed | us-east-1c        |

**Inbound rules (5)**

| Name | Security group rule ID | IP version | Type       | Protocol |
|------|------------------------|------------|------------|----------|
| -    | sgr-03162f929687b67f0  | IPv4       | HTTPS      | TCP      |
| -    | sgr-0cb9d448eb300ecd2  | IPv4       | SSH        | TCP      |
| -    | sgr-0a80a4e3ff9234ab0  | IPv4       | Custom TCP | TCP      |
| -    | sgr-011ef665d601bba81  | IPv4       | Custom TCP | TCP      |
| -    | sgr-01f36695b58ecc8e5  | IPv4       | HTTP       | TCP      |

## Step 2:

```
ubuntu@ip-172-31-17-143:~$ sudo systemctl enable docker
ubuntu@ip-172-31-17-143:~$ sudo systemctl start docker
ubuntu@ip-172-31-17-143:~$ docker --version
Docker version 29.1.3, build 29.1.3-0ubuntu4.1
ubuntu@ip-172-31-17-143:~$
```

## Step 3:

```
ubuntu@ip-172-31-17-143:~$ curl -LO "https://dl.k8s.io/release/$(curl -L -s https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl"
chmod +x kubectl
sudo mv kubectl /usr/local/bin/
% Total % Received % Xferd Average Speed Time Time Time Current
Dload Upload Total Spent Left Speed
100 56.79M 100 56.79M 0 0 377.0M 0
ubuntu@ip-172-31-17-143:~$ kubectl version --client
Client Version: v1.36.2
Kustomize Version: v5.8.1
ubuntu@ip-172-31-17-143:~$
```

```

ubuntu@ip-172-31-17-143:~$ curl -LO https://storage.googleapis.com/minikube/releases/latest/minikube-linux-amd64
% Total    % Received % Xferd Average Speed   Time    Time     Time  Current
           Dload  Upload   Total   Spent    Left     Speed
100 128.6M 100 128.6M    0     0  244.6M    0     0      0
ubuntu@ip-172-31-17-143:~$ sudo install minikube-linux-amd64 /usr/local/bin/minikub
ubuntu@ip-172-31-17-143:~$ minikube version
minikube: command not found
ubuntu@ip-172-31-17-143:~$ sudo install minikube-linux-amd64 /usr/local/bin/minikube
ubuntu@ip-172-31-17-143:~$ minikube version
minikube version: v1.38.1
commit: c93a4cb9311efc66b90d33ea03f75f2c4120e9b0
ubuntu@ip-172-31-17-143:~$

```

## Starting the cluster

```

ubuntu@ip-172-31-17-143:~$ minikube start --driver=docker
* minikube v1.38.1 on Ubuntu 26.04
* Using the docker driver based on user configuration
! Starting v1.39.0, minikube will default to "containerd" container runtime. See #21973 for more info.

X The requested memory allocation of 3072MiB does not leave room for system overhead (total system memory: 3831MiB). You may
ace stability issues.
* Suggestion: Start minikube with less memory allocated: 'minikube start --memory=3072mb'

* Using Docker driver with root privileges
* Starting "minikube" primary control-plane node in "minikube" cluster
* Pulling base image v0.0.50 ...
* Downloading Kubernetes v1.35.1 preload ...
  > preloaded-images-k8s-v18-v1...: 272.45 MiB / 272.45 MiB 100.00% 285.35
  > gcr.io/k8s-minikube/kicbase...: 519.58 MiB / 519.58 MiB 100.00% 122.84
* Creating docker container (CPUs=2, Memory=3072MB) ...
* Preparing Kubernetes v1.35.1 on Docker 29.2.1 ...
* Configuring bridge CNI (Container Networking Interface) ...
* Verifying Kubernetes components...
  - Using image gcr.io/k8s-minikube/storage-provisioner:v5
* Enabled addons: storage-provisioner, default-storageclass
* Done! kubectl is now configured to use "minikube" cluster and "default" namespace by default
ubuntu@ip-172-31-17-143:~$

ubuntu@ip-172-31-17-143:~$ kubectl get nodes
NAME        STATUS    ROLES    AGE   VERSION
minikube    Ready     control-plane 26s   v1.35.1
ubuntu@ip-172-31-17-143:~$

```

```

ubuntu@ip-172-31-17-143:~$ minikube addons enable metrics-server
* metrics-server is an addon maintained by Kubernetes. For any concerns contact minikube on GitHub.
You can view the list of minikube maintainers at: https://github.com/kubernetes/minikube/blob/master/OWNERS
  - Using image registry.k8s.io/metrics-server/metrics-server:v0.8.1
* The 'metrics-server' addon is enabled
ubuntu@ip-172-31-17-143:~$

```

## Create Namespace food app

```
ubuntu@ip-172-31-17-143:~$ kubectl create namespace food-app
namespace/food-app created
ubuntu@ip-172-31-17-143:~$ kubectl get ns
NAME                STATUS   AGE
default             Active   2m5s
food-app            Active   45s
kube-node-lease     Active   2m5s
kube-public         Active   2m5s
kube-system         Active   2m5s
ubuntu@ip-172-31-17-143:~$
```

## Namespace.yaml

```
ubuntu@ip-172-31-17-143:~/food-ordering-k8s/yaml$ cat namespace.yaml
apiVersion: v1
kind: Namespace

metadata:
  name: food-app
ubuntu@ip-172-31-17-143:~/food-ordering-k8s/yaml$
```

```
ubuntu@ip-172-31-17-143:~/food-ordering-k8s/yaml$ kubectl get namespaces
NAME                STATUS   AGE
default             Active   10m
food-app            Active   9m4s
kube-node-lease     Active   10m
kube-public         Active   10m
kube-system         Active   10m
ubuntu@ip-172-31-17-143:~/food-ordering-k8s/yaml$
```

## Cnfigmap.yaml

```
ubuntu@ip-172-31-17-143:~/food-ordering-k8s/yaml$ cat configmap.yaml
apiVersion: v1
kind: ConfigMap

metadata:
  name: app-config
  namespace: food-app

data:
  DB_HOST: mysql-service
  DB_PORT: "3306"
  APP_ENV: production
ubuntu@ip-172-31-17-143:~/food-ordering-k8s/yaml$
```

```
ubuntu@ip-172-31-17-143:~/food-ordering-k8s/yaml$ kubectl apply -f configmap.yaml
configmap/app-config created
```

```
ubuntu@ip-172-31-17-143:~/food-ordering-k8s/yaml$ kubectl get configmap -n food-app
NAME          DATA      AGE
app-config    3          27s
kube-root-ca.crt 1          11m
ubuntu@ip-172-31-17-143:~/food-ordering-k8s/yaml$ kubectl describe configmap app-config -n food-app
Name:         app-config
Namespace:    food-app
Labels:       <none>
Annotations:  <none>

Data
====
APP_ENV:
----
production

DB_HOST:
----
mysql-service

DB_PORT:
----
3306

BinaryData
====
Events: <none>
```

## Secret.yaml

```
ubuntu@ip-172-31-17-143:~/food-ordering-k8s/yaml$ touch secrets.yaml
ubuntu@ip-172-31-17-143:~/food-ordering-k8s/yaml$ nano secrets.yaml
ubuntu@ip-172-31-17-143:~/food-ordering-k8s/yaml$ cat secrets.yaml
apiVersion: v1
kind: Secret
metadata:
  name: mysql-secret
  namespace: food-app

type: Opaque

stringData:
  MYSQL_ROOT_PASSWORD: admin123
  MYSQL_DATABASE: foodb
ubuntu@ip-172-31-17-143:~/food-ordering-k8s/yaml$
```

```
ubuntu@ip-172-31-17-143:~/food-ordering-k8s/yaml$ kubectl apply -f secrets.yaml
secret/mysql-secret created
ubuntu@ip-172-31-17-143:~/food-ordering-k8s/yaml$
```

## My sql Deployment.yaml

```
ubuntu@ip-172-31-17-143:~/food-ordering-k8s/yaml$ cat mysql-deployment.yaml
apiVersion: apps/v1
kind: Deployment

metadata:
  name: mysql
  namespace: food-app

spec:
  replicas: 1

  selector:
    matchLabels:
      app: mysql

  template:
    metadata:
      labels:
        app: mysql

    spec:
      containers:
        - name: mysql
          image: mysql:8.0

          ports:
            - containerPort: 3306
```

```
3. 18.234.217.208 x +
template:
  metadata:
    labels:
      app: mysql

  spec:
    containers:
      - name: mysql
        image: mysql:8.0

        ports:
          - containerPort: 3306

        env:
          - name: MYSQL_ROOT_PASSWORD
            valueFrom:
              secretKeyRef:
                name: mysql-secret
                key: MYSQL_ROOT_PASSWORD

          - name: MYSQL_DATABASE
            valueFrom:
              secretKeyRef:
                name: mysql-secret
                key: MYSQL_DATABASE
```

my-service.yaml

```
ubuntu@ip-172-31-17-143:~/food-ordering-k8s/yaml$ cat mysql-service.yaml
apiVersion: v1
kind: Service

metadata:
  name: mysql-service
  namespace: food-app

spec:

  selector:
    app: mysql

  ports:

    - port: 3306

      targetPort: 3306

  type: ClusterIP
ubuntu@ip-172-31-17-143:~/food-ordering-k8s/yaml$
```

## app-deployment.yaml

```
ubuntu@ip-172-31-17-143:~/food-ordering-k8s/yaml$ cat app-deployment.yaml
apiVersion: apps/v1
kind: Deployment

metadata:
  name: food-app
  namespace: food-app

spec:

  replicas: 2

  selector:

    matchLabels:

      app: food-app

  template:

    metadata:

      labels:

        app: food-app

    spec:

      containers:
```

```
  labels:
    app: food-app
spec:
  containers:
  - name: food-app
    image: madhu934652/food-app:latest
    ports:
    - containerPort: 8080
    envFrom:
    - configMapRef:
        name: app-config
    resources:
      requests:
        cpu: "200m"
        memory: "256Mi"
      limits:
        cpu: "500m"
        memory: "512Mi"
```

ubuntu@ip-172-31-17-143:~/food-ordering-k8s/yaml\$ █

## app-service.yaml

```
ubuntu@ip-172-31-17-143:~/food-ordering-k8s/yaml$ cat app-service.yaml
apiVersion: v1
kind: Service
metadata:
  name: food-service
  namespace: food-app
spec:
  selector:
    app: food-app
  ports:
  - port: 80
    targetPort: 8080
  type: NodePort
ubuntu@ip-172-31-17-143:~/food-ordering-k8s/yaml$ █
```

```
kind: Ingress
metadata:
  name: food-ingress
  namespace: food-app
spec:
  rules:
  - host: food.local
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: food-service
            port:
              number: 80
```

## Hpa.yaml

```
kind: HorizontalPodAutoscaler
metadata:
  name: food-hpa
  namespace: food-app
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: food-app
  minReplicas: 2
  maxReplicas: 6
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
```



```
name: food-hpa
namespace: food-app
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: food-app
  minReplicas: 2
  maxReplicas: 6
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 50
ubuntu@ip-172-31-17-143:~/food-ordering-k8s/yaml$
```

## networkpolicy.yaml

```
ubuntu@ip-172-31-17-143:~/food-ordering-k8s/yaml$ cat networkpolicy.yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-app-to-db
  namespace: food-app
spec:
  podSelector:
    matchLabels:
      app: mysql
  ingress:
  - from:
    - podSelector:
        matchLabels:
          app: food-app
ubuntu@ip-172-31-17-143:~/food-ordering-k8s/yaml$
```

## resource-limit.yaml

```
ubuntu@ip-172-31-17-143:~/food-ordering-k8s/yaml$ cat resource-limit.yaml
apiVersion: v1

kind: LimitRange

metadata:
  name: cpu-memory-limit
  namespace: food-app

spec:
  limits:
  - default:
      cpu: "500m"
      memory: "512Mi"
    defaultRequest:
      cpu: "200m"
      memory: "256Mi"
  type: Container
ubuntu@ip-172-31-17-143:~/food-ordering-k8s/yaml$
```

kubectl apply -f namespace.yaml

kubectl apply -f secret.yaml

kubectl apply -f configmap.yaml

kubectl apply -f mysql-deployment.yaml

kubectl apply -f mysql-service.yaml

kubectl apply -f app-deployment.yaml

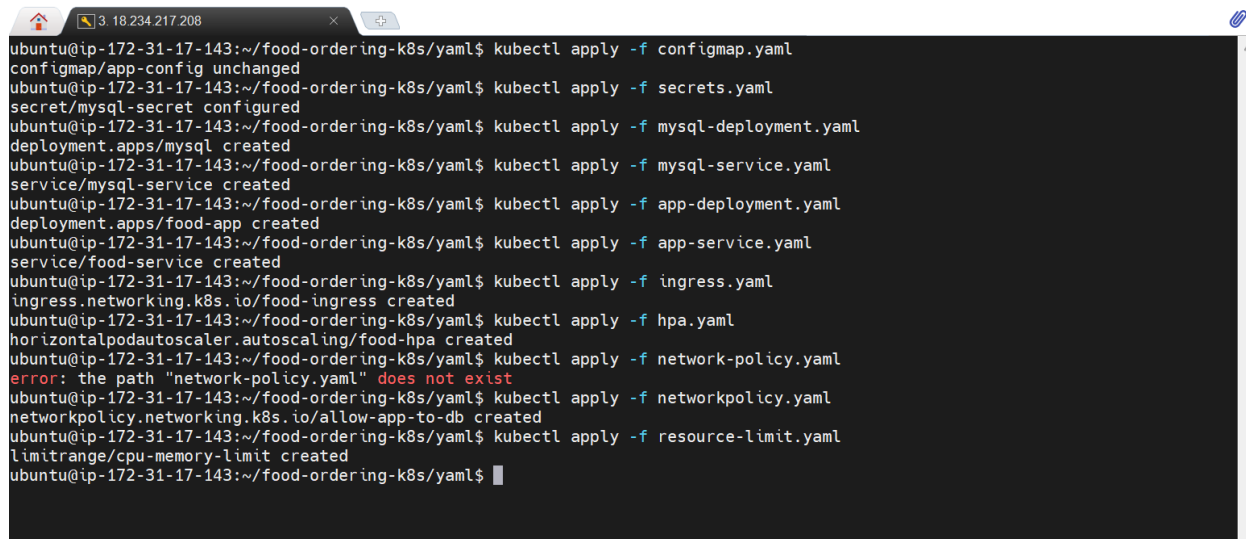
kubectl apply -f app-service.yaml

kubectl apply -f ingress.yaml

kubectl apply -f hpa.yaml

kubectl apply -f network-policy.yaml

kubectl apply -f resource-limit.yaml

A terminal window with a dark background and light text. The window title bar shows a home icon, a network icon, and the IP address '3.18.234.217.208'. The terminal content shows a series of kubectl commands being executed in a directory named '/food-ordering-k8s/yaml'. The commands and their outputs are: 1. 'kubectl apply -f configmap.yaml' results in 'configmap/app-config unchanged'. 2. 'kubectl apply -f secrets.yaml' results in 'secret/mysql-secret configured'. 3. 'kubectl apply -f mysql-deployment.yaml' results in 'deployment.apps/mysql created'. 4. 'kubectl apply -f mysql-service.yaml' results in 'service/mysql-service created'. 5. 'kubectl apply -f app-deployment.yaml' results in 'deployment.apps/food-app created'. 6. 'kubectl apply -f app-service.yaml' results in 'service/food-service created'. 7. 'kubectl apply -f ingress.yaml' results in 'ingress.networking.k8s.io/food-ingress created'. 8. 'kubectl apply -f hpa.yaml' results in 'horizontalpodautoscaler.autoscaling/food-hpa created'. 9. 'kubectl apply -f network-policy.yaml' results in an error: 'error: the path "network-policy.yaml" does not exist'. 10. 'kubectl apply -f networkpolicy.yaml' results in 'networkpolicy.networking.k8s.io/allow-app-to-db created'. 11. 'kubectl apply -f resource-limit.yaml' results in 'limitrange/cpu-memory-limit created'. The prompt 'ubuntu@ip-172-31-17-143:~/food-ordering-k8s/yaml\$' is visible at the end of the last command.

```
ubuntu@ip-172-31-17-143:~/food-ordering-k8s/yaml$ kubectl apply -f configmap.yaml
configmap/app-config unchanged
ubuntu@ip-172-31-17-143:~/food-ordering-k8s/yaml$ kubectl apply -f secrets.yaml
secret/mysql-secret configured
ubuntu@ip-172-31-17-143:~/food-ordering-k8s/yaml$ kubectl apply -f mysql-deployment.yaml
deployment.apps/mysql created
ubuntu@ip-172-31-17-143:~/food-ordering-k8s/yaml$ kubectl apply -f mysql-service.yaml
service/mysql-service created
ubuntu@ip-172-31-17-143:~/food-ordering-k8s/yaml$ kubectl apply -f app-deployment.yaml
deployment.apps/food-app created
ubuntu@ip-172-31-17-143:~/food-ordering-k8s/yaml$ kubectl apply -f app-service.yaml
service/food-service created
ubuntu@ip-172-31-17-143:~/food-ordering-k8s/yaml$ kubectl apply -f ingress.yaml
ingress.networking.k8s.io/food-ingress created
ubuntu@ip-172-31-17-143:~/food-ordering-k8s/yaml$ kubectl apply -f hpa.yaml
horizontalpodautoscaler.autoscaling/food-hpa created
ubuntu@ip-172-31-17-143:~/food-ordering-k8s/yaml$ kubectl apply -f network-policy.yaml
error: the path "network-policy.yaml" does not exist
ubuntu@ip-172-31-17-143:~/food-ordering-k8s/yaml$ kubectl apply -f networkpolicy.yaml
networkpolicy.networking.k8s.io/allow-app-to-db created
ubuntu@ip-172-31-17-143:~/food-ordering-k8s/yaml$ kubectl apply -f resource-limit.yaml
limitrange/cpu-memory-limit created
ubuntu@ip-172-31-17-143:~/food-ordering-k8s/yaml$
```

Now verify everything

```
ubuntu@ip-172-31-17-143:~/food-ordering-k8s/yaml$ kubectl describe configmap app-config -n food-app
Name:         app-config
Namespace:    food-app
Labels:       <none>
Annotations:  <none>

Data
====
APP_ENV:
----
production

DB_HOST:
----
mysql-service

DB_PORT:
----
3306

BinaryData
====

Events: <none>
ubuntu@ip-172-31-17-143:~/food-ordering-k8s/yaml$
```

## Secrets

```
ubuntu@ip-172-31-17-143:~/food-ordering-k8s/yaml$ kubectl get secret -n food-app
NAME          TYPE   DATA   AGE
mysql-secret   Opaque 2       19m
ubuntu@ip-172-31-17-143:~/food-ordering-k8s/yaml$ kubectl describe secret mysql-secret -n food-app
Name:         mysql-secret
Namespace:    food-app
Labels:       <none>
Annotations:  <none>

Type: Opaque

Data
====
MYSQL_DATABASE:      6 bytes
MYSQL_ROOT_PASSWORD: 8 bytes
ubuntu@ip-172-31-17-143:~/food-ordering-k8s/yaml$
```

Here I got imagepullbackoff error due to the wrong name make sure you have given container name correct and image name correct

```
ubuntu@ip-172-31-17-143:~/food-ordering-k8s/yaml$ kubectl get deployment -n food-app
NAME          READY   UP-TO-DATE   AVAILABLE   AGE
food-app      0/2     2             0           6m15s
mysql         1/1     1             1           6m33s
ubuntu@ip-172-31-17-143:~/food-ordering-k8s/yaml$ kubectl get pods -n food-app
NAME                                READY   STATUS             RESTARTS   AGE
food-app-795d789647-qprh5           0/1     ImagePullBackOff    0           6m33s
food-app-795d789647-t7rgl           0/1     ImagePullBackOff    0           6m33s
mysql-6db98f76cc-fxmbg              1/1     Running             0           6m51s
ubuntu@ip-172-31-17-143:~/food-ordering-k8s/yaml$
```

```
ubuntu@ip-172-31-33-155:~$ kubectl get all -n food-app
```

| NAME                              | READY | STATUS                     | RESTARTS | AGE   |
|-----------------------------------|-------|----------------------------|----------|-------|
| pod/cafe-website-74598fc9f9-wccfb | 1/1   | Running                    | 0        | 5m13s |
| pod/cafe-website-74598fc9f9-xvh5l | 1/1   | Running                    | 0        | 5m13s |
| pod/mysql-6db98f76cc-5nz28        | 0/1   | CreateContainerConfigError | 0        | 16m   |

| NAME                 | TYPE     | CLUSTER-IP     | EXTERNAL-IP | PORT(S)      | AGE  |
|----------------------|----------|----------------|-------------|--------------|------|
| service/food-service | NodePort | 10.109.148.178 | <none>      | 80:30993/TCP | 5m6s |

| NAME                         | READY | UP-TO-DATE | AVAILABLE | AGE   |
|------------------------------|-------|------------|-----------|-------|
| deployment.apps/cafe-website | 2/2   | 2          | 2         | 5m13s |
| deployment.apps/mysql        | 0/1   | 1          | 0         | 16m   |

| NAME                                    | DESIRED | CURRENT | READY | AGE   |
|-----------------------------------------|---------|---------|-------|-------|
| replicaset.apps/cafe-website-74598fc9f9 | 2       | 2       | 2     | 5m13s |
| replicaset.apps/mysql-6db98f76cc        | 1       | 1       | 0     | 16m   |

| NAME                                         | REFERENCE           | TARGETS            | MINPODS | MAXPODS | REPLICAS | AGE |
|----------------------------------------------|---------------------|--------------------|---------|---------|----------|-----|
| horizontalpodautoscaler.autoscaling/food-hpa | Deployment/food-app | cpu: <unknown>/50% | 2       | 6       | 0        | 88s |

```
ubuntu@ip-172-31-33-155:~$
```

```
ubuntu@ip-172-31-33-155:~$ kubectl apply -f secret.yaml
secret/mysql-secret created
ubuntu@ip-172-31-33-155:~$ kubectl rollout restart deployment mysql -n food-app
deployment.apps/mysql restarted
ubuntu@ip-172-31-33-155:~$ kubectl get pods -n food-app
```

| NAME                          | READY | STATUS      | RESTARTS | AGE |
|-------------------------------|-------|-------------|----------|-----|
| cafe-website-74598fc9f9-wccfb | 1/1   | Running     | 0        | 10m |
| cafe-website-74598fc9f9-xvh5l | 1/1   | Running     | 0        | 10m |
| mysql-645b6f7cf-vgqz6         | 1/1   | Running     | 0        | 12s |
| mysql-6db98f76cc-5nz28        | 1/1   | Terminating | 0        | 21m |

```
ubuntu@ip-172-31-33-155:~$
```

```
ubuntu@ip-172-31-33-155:~$ kubectl get secrets -n food-app
No resources found in food-app namespace.
ubuntu@ip-172-31-33-155:~$ kubectl apply -f secret.yaml
secret/mysql-secret created
ubuntu@ip-172-31-33-155:~$ kubectl rollout restart deployment mysql -n food-app
deployment.apps/mysql restarted
ubuntu@ip-172-31-33-155:~$ kubectl get pods -n food-app
```

| NAME                          | READY | STATUS      | RESTARTS | AGE |
|-------------------------------|-------|-------------|----------|-----|
| cafe-website-74598fc9f9-wccfb | 1/1   | Running     | 0        | 10m |
| cafe-website-74598fc9f9-xvh5l | 1/1   | Running     | 0        | 10m |
| mysql-645b6f7cf-vgqz6         | 1/1   | Running     | 0        | 12s |
| mysql-6db98f76cc-5nz28        | 1/1   | Terminating | 0        | 21m |

```
ubuntu@ip-172-31-33-155:~$ curl ifconfig.me
54.158.57.71ubuntu@ip-172-31-33-155:~$ ^C
ubuntu@ip-172-31-33-155:~$ kubectl get svc -n food-app
```

| NAME         | TYPE     | CLUSTER-IP     | EXTERNAL-IP | PORT(S)      | AGE |
|--------------|----------|----------------|-------------|--------------|-----|
| food-service | NodePort | 10.109.148.178 | <none>      | 80:30993/TCP | 11m |

```
ubuntu@ip-172-31-33-155:~$ ^C
ubuntu@ip-172-31-33-155:~$ kubectl get endpoints -n food-app
Warning: v1 Endpoints is deprecated in v1.33+; use discovery.k8s.io/v1 EndpointSlice
```

| NAME         | ENDPOINTS | AGE |
|--------------|-----------|-----|
| food-service | <none>    | 12m |

```
ubuntu@ip-172-31-33-155:~$
```

```

ubuntu@ip-172-31-33-155:~$ kubectl get endpoints -n food-app
Warning: v1 Endpoints is deprecated in v1.33+; use discovery.k8s.io/v1 EndpointSlice
NAME      ENDPOINTS          AGE
pod-service 10.244.0.4:80,10.244.0.5:80 15m
ubuntu@ip-172-31-33-155:~$ ^C
ubuntu@ip-172-31-33-155:~$ kubectl get svc -n food-app
NAME      TYPE        CLUSTER-IP      EXTERNAL-IP      PORT(S)          AGE
pod-service NodePort    10.109.148.178  <none>           80:30993/TCP     16m
ubuntu@ip-172-31-33-155:~$

```

Here the port is 3099 now add the port in the security groups

```

food-service NodePort 10.109.148.178 <none> 80:30993/TCP 16m
ubuntu@ip-172-31-33-155:~$ ^C
ubuntu@ip-172-31-33-155:~$ curl http://localhost:30993
curl: (7) Failed to connect to localhost port 30993 after 0 ms: Could not connect to server
ubuntu@ip-172-31-33-155:~$ ^C
ubuntu@ip-172-31-33-155:~$

```

Here we can access the website

```

ubuntu@ip-172-31-33-155:~$ minikube ip
192.168.49.2
ubuntu@ip-172-31-33-155:~$ ^C
ubuntu@ip-172-31-33-155:~$ ^C
ubuntu@ip-172-31-33-155:~$ ^C
ubuntu@ip-172-31-33-155:~$ minikube service food-service -n food-app --url
http://192.168.49.2:30993
ubuntu@ip-172-31-33-155:~$ ^C
ubuntu@ip-172-31-33-155:~$ curl http://192.168.49.2:30993
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Brew & Bliss Café</title>
  <link rel="stylesheet" href="styles.css">
</head>

```

Port forwarding because it cant be accessed in the web because of node port

```

ubuntu@ip-172-31-33-155:~$ kubectl port-forward -n food-app service/food-service 8080:80 --address 0.0.0.0
Forwarding from 0.0.0.0:8080 -> 80
Handling connection for 8080
^Cubuntu@ip-172-31-33-155:~$

```

Now enter your ec2 ip with http you will see the website in your web browser

